

Ring of the Middle Earth

Architecture Document - Option B Go Implementation

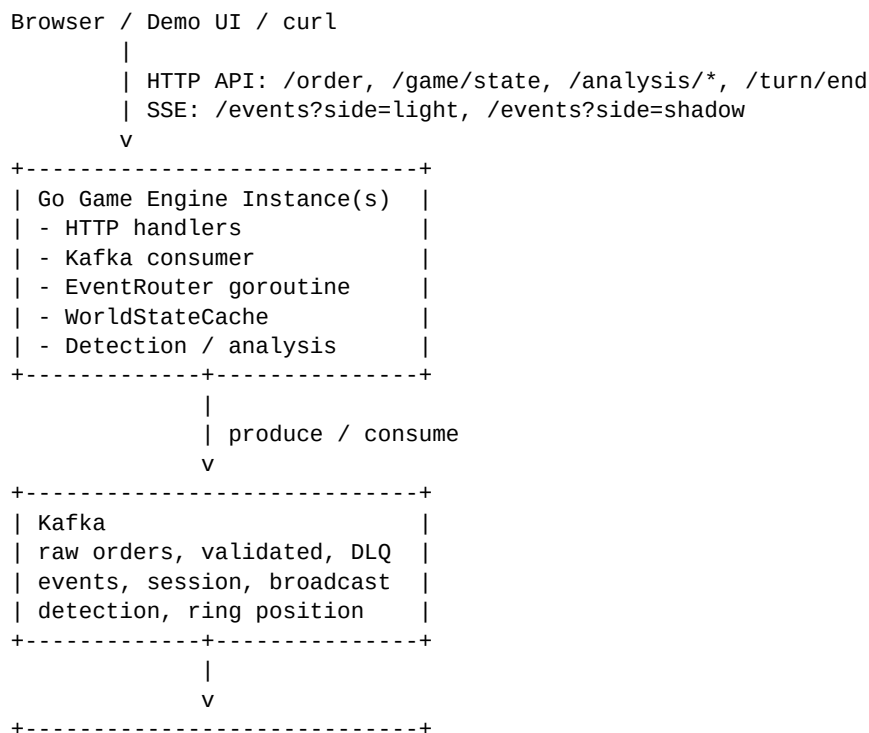
Kafka + Go + SSE + Demo UI

Purpose. This document explains the architecture of the Option B Go implementation, including running services, goroutines, Kafka topics, state handling, fault tolerance, transactional GameOver behavior, tests, and design tradeoffs.

Section	Content
System diagram	Running services, connections, and data flow
Go-specific architecture	Goroutine map, channels, buffer capacities, and termination conditions
Kafka diagram	All 10 topics, producers, consumers, partition keys, and cleanup policies
Paradigm justification	Why Go + Kafka fits, what is harder, and how Akka would solve the hardest parts
Reflection	Minimum 300-word reflection and design lessons
Appendix	Testing evidence, demo commands, and LLM usage log

1. System Diagram

The system is built around an event-driven Go game engine and Kafka. The browser or curl client sends HTTP requests to the Go API. Validated commands and game events flow through Kafka before being applied to the game state or streamed to clients through SSE.



```
| Schema Registry |
| Avro schemas and evolution |
+-----+
```

In local development, a single Go server can be started with `go run`. For the distributed demo, the same Go engine is packaged into a Docker image and scaled to three instances in the same Kafka consumer group.

2. Running Services

Service	Role	Connection / Notes
Kafka	Event backbone	Stores orders, validation results, broadcast events, session state, detection events, and DLQ records
Schema Registry	Schema management	Stores Avro subjects and supports schema evolution demo
Go game-engine x3	Stateless application tier	Consumes Kafka as one consumer group; local state is reconstructed from config and events
Demo UI	Browser interface	Served by Go server from the /ui static directory
SSE clients	Live event streams	Light and Shadow sides receive filtered event streams

3. Option B Go Goroutine Map

The Go implementation uses explicit goroutines and channels. The main event path is Kafka consumer -> eventCh -> EventRouter -> game logic / SSE channels / cache updates.

```
main goroutine
- loads config
- initializes WorldStateCache
- initializes Kafka producers
- starts HTTP server
- starts worker goroutines

KafkaConsumer goroutine
input: Kafka topics
output: eventCh (buffered channel, cap approx. 100)
termination: process shutdown or consumer close

EventRouter goroutine
input: eventCh
output: lightSideSSECh, darkSideSSECh, updateCh
responsibilities:
- route raw orders to validation
- route validated orders to processor/actions
- route DLQ events
- strip Ring Bearer position before Dark Side delivery
- broadcast WorldStateSnapshot events
termination: eventCh close or process shutdown

CacheManager goroutine
input: updateCh
output: WorldStateCache mutation
termination: updateCh close or process shutdown

SSE handler goroutines
input: lightSideSSECh or darkSideSSECh
```

output: HTTP text/event-stream
termination: client disconnect

HTTP handler goroutines
input: HTTP requests
output: HTTP responses and Kafka messages
termination: request completion

Channel / Flow	Producer	Consumer	Purpose
eventCh	KafkaConsumer	EventRouter	Central event fan-in channel
updateCh	EventRouter / processors	CacheManager	State update channel
lightSideSSECh	EventRouter	Light SSE clients	Unstripped Light Side event stream
darkSideSSECh	EventRouter	Dark SSE clients	Filtered Dark Side event stream

4. Kafka Diagram and Topic Rationale

Kafka decouples order ingestion, validation, state updates, event streaming, analysis, and fault tolerance. The final architecture uses ten topics. The game.session topic is log-compacted and stores the latest session-level state, while game.broadcast uses delete cleanup for short-lived UI snapshots.

Topic	Partition key	Partitions	Cleanup	Produced by	Consumed by
game.orders.raw	playerId	3	delete	HTTP /order	Validation topology
game.orders.validated	unitId	6	delete	Validation topology	Game engine processor
game.events.unit	unitId	6	delete	Game engine	State/cache consumers
game.events.region	regionId	6	delete	Game engine	State/cache consumers
game.events.path	pathId	6	delete	Game engine	State/cache consumers
game.session	session/gameId	1	compact	Game engine	Recovery/session readers
game.broadcast	none	1	delete	Game engine	SSE/EventRouter
game.ring.position	none	1	delete	Game engine	Light SSE only
game.ring.detection	playerId	2	delete	Detection system	Dark SSE only
game.dlq	errorCode	3	delete	Validation topology	Debug/demo consumers

HTTP /order -> game.orders.raw
Validation rules -> game.orders.validated or game.dlq
Game engine -> game.events.unit / game.events.region / game.events.path
Turn end -> game.broadcast
Ring movement -> game.ring.position (Light Side only)
Detection -> game.ring.detection (Dark Side only)
Session state -> game.session (compacted recovery topic)

5. State Management and Information Hiding

The Go layer keeps a WorldStateCache for fast HTTP and SSE responses. The Light Side receives the Ring Bearer true region. The Dark Side receives only detection information and last detected region; true Ring Bearer position is removed before the response leaves the server.

Path	Light Side	Dark Side
GET /game/state	Ring Bearer region visible	Ring Bearer region stripped
game.broadcast SSE	Unstripped snapshot	Filtered snapshot through stripRingBearer

game.ring.position	Delivered	Not delivered
game.ring.detection	Not primary target	Delivered as detection evidence

6. Validation and Analysis Pipelines

6.1 Validation and DLQ

The validation layer checks invalid orders before they reach the game processor. Rejected records are published to game.dlq with a stable error code and the original raw payload.

Implemented error codes:

- WRONG_TURN
- NOT_YOUR_UNIT
- PATH_BLOCKED
- INVALID_PATH
- UNIT_NOT_ADJACENT
- INVALID_TARGET
- ABILITY_ON_COOLDOWN
- DUPLICATE_UNIT_ORDER

6.2 Route Risk Pipeline

Pipeline 1 ranks candidate Ring Bearer routes for the Light Side. It computes risk by combining region threat level, path surveillance, blocked/threatened path status, and Nazgul proximity. Validated records are enriched with routeRiskScore, threatenedPaths, and blockedPaths.

6.3 Interception Pipeline

Pipeline 2 creates Shadow Side interception plans. It compares Nazgul movement distance to the Ring Bearer route progress and computes an interception score.

7. Gameplay Mechanics

Requirement	Implementation
Detection	Hidden until early turns pass; Nazgul DetectionRange is read from config; Sauron passive amplifier increases effective range
Maia dispatch	The order type remains MAIA_ABILITY. The effect is selected by UnitConfig.AbilityEffect, not unit ID
Path blocking	A blocked path reverts to OPEN when the blocking unit leaves the path endpoint
Combat	Base strength plus modifiers such as fortified region, controlled region, leadership, and UrukHai attack behavior
GameOver	Transactional Kafka producer demonstrates aborted vs committed GameOver records with read_committed consumer

8. Fault Tolerance and Multi-Instance Execution

The distributed demo runs three Go instances in the same Kafka consumer group. Each instance is stateless from the application-tier perspective. Kafka assigns partitions to instances. If one instance stops, Kafka revokes and reassigns partitions to the remaining instances. When the stopped instance returns, it rejoins the group and receives assignments again.

```
docker compose up -d --build --scale game-engine=3 game-engine
```

```
docker compose ps game-engine
```

```
docker stop ring-of-the-middle-earth-game-engine-3
```

```
docker compose logs -f game-engine
```

Expected evidence:

Consumer group rebalance: partitions revoked

Consumer group rebalance: partitions assigned

Each Go instance uses a unique transactional.id derived from its Docker container hostname. This avoids transactional producer fencing between active instances.

9. Goroutine Leak Check

The B9 evidence is collected using net/http/pprof. After starting the server, the goroutine profile is checked before and after ten turn transitions. The expected result is a stable or nearly stable goroutine count.

```
curl -s http://localhost:8080/debug/pprof/goroutine?debug=1 | grep "goroutine profile"
```

```
for i in {1..10}; do
  curl -s -X POST http://localhost:8080/turn/end
  echo
done
```

```
curl -s http://localhost:8080/debug/pprof/goroutine?debug=1 | grep "goroutine profile"
```

10. Paradigm Justification

10.1 Why Go + Kafka is well-suited

Go and Kafka fit this problem because the game is naturally event-driven. Orders, validation results, state changes, detection events, and broadcast snapshots can be represented as immutable messages. Go provides lightweight goroutines and simple channels for concurrent routing inside each engine instance. Kafka provides durable, ordered logs and consumer group rebalancing for distributed execution. This separation makes the Go instances interchangeable: they can be scaled horizontally, stopped, and restarted while Kafka remains the backbone of the system.

10.2 What is harder in Go + Kafka than in Akka

The hardest part in Go + Kafka is coordinating state consistency and lifecycle management manually. Akka actors naturally own state and provide supervision strategies, message mailboxes, and cluster sharding abstractions. In Go, we must explicitly define goroutines, channels, locking, event routing, visibility filtering, state cache updates, and consumer group behavior. Transactional GameOver and exactly-once style behavior also require careful Kafka producer configuration rather than being hidden behind a higher-level actor lifecycle.

10.3 How Akka would solve the hardest parts

Akka would likely model units, paths, regions, and world state as actors. UnitActor and PathActor would encapsulate state transitions, while supervision strategies would define restart and recovery behavior. Cluster sharding would distribute actors across nodes and rebalance shards when a node fails. The two hardest parts of this Go implementation - state recovery and distributed fault tolerance - would be handled through actor persistence, supervision, and sharding rather than explicit Kafka consumer group callbacks and manually maintained WorldStateCache logic.

11. Test Evidence

Command / Test	Purpose
----------------	---------

go test ./...	Runs pipeline, validation, combat, path blocking, state visibility, and router tests
go test -race ./...	Checks for race conditions in test-covered paths
./test/kafka_topics_check.sh	K1 topic describe evidence
./test/schema_evolution_v1_consumer.sh	K3 V1 consumer compatibility with V2 fields
./test/game_over_transaction_demo.sh demo-k6-1	K6 transactional GameOver demonstration

12. Reflection

The most difficult part of this project was not implementing a single endpoint or a single rule, but connecting many moving parts into one coherent distributed system. Kafka validation, route analysis, turn processing, event routing, information hiding, detection, SSE delivery, and transactional GameOver behavior all interact with each other. A small change in one layer can affect tests, UI behavior, or demo output in another layer. Another unexpected challenge was information hiding. It was not enough to hide the Ring Bearer only in one endpoint; the position also had to be removed from Dark Side SSE payloads and copied state structures without mutating the original Light Side state. This required careful deep-copy logic and router tests.

The second difficult area was fault tolerance. Running three Go instances is easy, but proving that Kafka actually reassigns partitions and that each instance has a safe transactional producer identity requires explicit logging and Docker-based testing. Compared with a single-process application, the distributed version needs more operational evidence: topic descriptions, consumer group rebalance logs, schema compatibility checks, pprof output, and transactional consumer output.

If I redesigned the project from the beginning, I would separate the repository layout more cleanly. The Docker Compose file, configuration files, schemas, Go engine, and documentation should all live under one version-controlled root. I would also implement session recovery earlier by treating game.session as the central compacted session topic from the first iteration. Finally, I would define all gameplay effects, including Maia abilities, path effects, combat modifiers, and detection modifiers, in config before writing logic. This would reduce hardcoded assumptions and make Q&A/code review stronger. Overall, the Go + Kafka approach was a good fit because it forced a clear event-driven design, but it also required careful documentation and testing to prove correctness.

13. Appendix - LLM Usage Log

The following log summarizes how AI assistance was used. The final code, tests, configuration changes, and design decisions were reviewed, modified, and executed by the project team.

Interaction / Area	Prompt / Need	Used Result	Changed or Rejected
Kafka validation and DLQ	Asked for help implementing and testing validation error codes	Used suggested validation structure and DLQ payload format	Adjusted error codes and curl cases based on actual logs
WorldState and information hiding	Asked how to hide Ring Bearer region from Dark Side	Used deep-copy and stripRingBearer approach	Fixed mutation bug after manual curl testing
Detection system	Asked for hidden-until-turn and Nazgul detection guidance	Used detection flow and Sauron amplifier idea	Verified with turn-end logs
HTTP API and UI	Asked for missing endpoints and basic UI	Used simple HTML/CSS/JS demo panel	Modified based on actual browser and API behavior
Maia dispatch	Asked how to make MaiaAbility safer for Q&A	Used config-driven AbilityEffect dispatch	Changed from unit ID switch to config field
Testing	Asked for Go tests for pipeline, combat, validation, router, and path blocking	Used generated test skeletons	Fixed config path and JSON issues locally
Kafka demos	Asked for topic describe, schema evolution, and GameOver exactly-once demo	Used shell scripts and transactional producer guidance	Adapted Dockerfile and transactional.id per container

Documentation	Asked for README and architecture documentation help	Used structure and wording as a basis	Edited formatting and scope for final submission
---------------	--	---------------------------------------	--

14. Final Demo Checklist

- Start Kafka stack and verify all topics, including game.session.
- Run the Go server locally and show /health.
- Show Light Side state vs Shadow Side hidden Ring Bearer state.
- Show route and interception analysis endpoints or UI panels.
- Run MaiaAbility for Gandalf and Saruman and explain AbilityEffect dispatch.
- Run path blocking/revert test or show passing path_blocking_test.go.
- Run go test ./... and go test -race ./...
- Run three game-engine instances and show Kafka rebalance logs.
- Run transactional GameOver demo and show only committed GameOver appears.
- Show pprof goroutine profile before and after ten turns.
- Open and explain this architecture PDF.